

Self-supervised learning

הרעיון הוא היכולת ללמוד בלי תגיות על הנתונים. הבעיה בלמידה מונחית היא שהיא דורשת המון נתונים, והרבה פעמים קשה לאסוף כמויות כאלה של נתונים:

- לפעמים מאוד קשה לתייג אותם, כי זה דורש אנשים מומחים (למשל - רדיולוגיה)
- לפעמים פשוט אין מספיק נתונים (למשל - מחלות נדירות)
- לפעמים המומחים שנותנים את התוויות לא מסכימים על התוויות.
- רעיון נפוץ זה לאסוף את הנתונים והתוויות מהרשת - אבל המידע באינטרנט לא תמיד נכון, וגם בד"כ אם יש טקסט ליד תמונה הטקסט לא אומר מה יש בתמונה אלא נותן קונטקסט (שלא עוזר לנו למשימת הלמידה המונחית)

אם ניקח דוגמה מתינוקות שלומדים - רוב הלמידה שלהם בהתחלה היא לא מתוויגת. איך אנחנו יכולים לעשות משהו דומה? ללמוד בלי תיוגים? חלק גדול ממה שהרשת עושה זה ללמוד ייצוג, ורק בסוף שמים מסווג על הייצוג שמצאנו. זה אומר שאנחנו לומדים שני דברים - גם את המסווג וגם את הייצוג. ראינו שהרבה פעמים הייצוגים הם אוניברסליים, ולא קשורים הדוק לסיווג. אז יש פה תקווה שהייצוג שאנחנו לומדים שימושי לעוד דברים.

דרך אחת להשתמש בזה היא transfer learning. הרעיון - אחרי שאימנו רשת על כמה מחלקות, מקפידים את רוב השכבות ומאמנים רק את השכבה האחרונה, של המסווגים. בגלל שיש הרבה פחות פרמטרים (רק שכבה אחת), צריך הרבה פחות נתונים בשביל לאמן את הרשת.

לפי Yann LeCun, לסיגנל של supervision יש 3 חלקים:

1. העיקר זה האינפורמציה הוויזואלית על העולם שנמצאת בתמונה.
2. רכיב של supervised learning - התוויות. כמות האינפורמציה כאן היא מוגבלת.
3. רכיב של reinforcement learning - תגמול שמקבלים על פעולות, וצריך ללמוד באמצעותו מתוך מספר קטן של פעולות.

הרכיב הראשון זה self-supervised learning.

הרעיון: לפתור בעיית supervised learning כשיש לנו מידע לא מתוויג. בשביל לעשות את זה משתמשים בחלק מהנתונים בתור תגיות.

תזכורת: בunsupervised learning, המטרה הייתה למצוא ייצוג טוב של הנתונים שתופס את המבנה וזורק דברים לא חשובים.

Autoencoders

הדרך לעשות את זה היא עם autoencoders, שזו רשת שבה הdata הוא הlabel של עצמו. אם השכבות הפנימיות הן ממימד יותר נמוך מאשר הנתונים, אז הן יהיו חייבות ללמוד איזשהו מבנה כדי שיוכלו לשחזר.

אפשר, בנוסף לזה, להוסיף שלב של corruption - ולאמן את הרשת לשחזר את data. המקורי, לפני הcorruption. זה מכריח את הרשת ללמוד את המבנה כדי שתוכל להתגבר על הcorruption. בעצם יש שני דברים שגורמים למודל ללמוד את המבנה:

- צוואר בקבוק - המימד של השכבות הפנימיות יותר נמוך. אבל זה לא מספיק, ולכן צריך גם:

- corruption, שהמודל נאלץ לנקות באמצעות למידה של המבנה.

זוהי הדרך שהמודלים המודרניים עובדים - קודם לומדים את המבנה עם self-supervised ורק אחרי זה עושים supervised ללמוד גם את התוויות. בלי זה רשתות עמוקות לא עובדות.

Spatial Relations

אפשר ליצור משימות אימון שבה input והdata הן מאותה דגימה. למשל - לקחת תמונה, לקחת באקראי שני אזורים, ולהשתמש באותו CNN (לפני הclassifier) כדי ללמוד איפה האזור השני נמצא לעומת האזור הראשון. אחרי שלומדים את זה - אפשר להשתמש ב-CNN הזה (שכבר למד ייצוג) בשביל ללמוד סיווגים אחרים. המודלים יכולים לרמות:

- הם יכולים ללמוד את הקצוות של האזורים במקום את התמונה, ולכן צריך לשים gap וגם להזיז קצת את האזורים (שלא יהיו grid סדור)

- הם יכולים ללמוד כל מיני דברים סטטיסטיים שלא באמת קשורים למבנה (למשל - לנחש שמשו נמצא למעלה בגלל שהוא נראה כמו שמייס). זה ערוץ של קורלציה שנמצא והמודל יכול למצוא אותו וללמוד משהו אחר ממה שהתכוונו.

איך בודקים איזה ייצוג נלמד?

בשביל להבין את הייצוג שלמדנו, אנחנו רוצים לייצג אותו חזותית.

גישות: 1. למפות את המיפוי לדו-מימד. למשל עם PCA. זה בעייתי בגלל שאנחנו מאבדים יותר מידי נתונים.

2. נשארים במימד הגבוה, ומחשבים לכל הdataset איפה הוא נמצא במימד הגבוה. מחשבים לכל תמונה מי השכנים שלה.

מסתבר שכשלוומדים self-supervised על spatial relations וכשלוומדים supervised, nearest neighbors נראה מאוד דומה.

עוד אפשרויות

1. ללמוד מודל לשחזר צבעים. אפשר להשיג נתונים בקלות - פשוט מורידים את הצבע מהתמונה.

2. אפשר גם סתם לחתוך או לעוות את התמונה.

3. אפשר לסובב את התמונה, ולבקש מהמודל ללמוד מה הכיוון הנכון.

טרמינולוגיה

כל השיטות השאלה הן self-supervised. ספציפית autoencoder זה השיטות בהן הפלט של הרשת שלומדת את הייצוג הוא הקלט המקורי (לפני עיוות)

Masked Autoencoders

- לוקחים תמונה.
 - מחלקים אותה לpatchים.
 - מסתירים חלק גדול מאוד מהpatchים (75% – 90%)
 - מאמנים encoder על הpatchים שנשארו, וdecoder שישחזר את התמונה.
 - זורקים את הdecoder ונשארים רק עם הencoder.
- זה עובד מאוד טוב, כי המימד של input יורד משמעותית כשזורקים patchים.

Contrastive Learning

עד עכשיו ראינו SSL¹ שהוא reconstruction-based. סוג אחר זה Constructive SSL. הרעיון: יוצרים מהתמונה המקורית שני עיוותים, ומלמדים רשת שתעבוד על שני העיוותים ותקטין את ההבדל ביניהם.

בעצם מגדירים את הloss בתור המרחק בין הייצוגים $\ell = \text{sim}(\mathbf{z}_i, \mathbf{z}_j)$. למשל - מרחק קוסיני. בד"כ נרצה לעשות $\exp\left(\frac{\text{sim}(\mathbf{z}_i, \mathbf{z}_j)}{\tau}\right)$. המודל יכול לרמות - למפות את הכל לאותו מקום, ואז תמיד נקבל $\ell = 0$. לכן נרצה איזה כוח דוחה שיאזן את הכוח המקרב הזה:

$$\ell_{i,j} = -\log \frac{\exp\left(\frac{\text{sim}(\mathbf{z}_i, \mathbf{z}_j)}{\tau}\right)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp\left(\frac{\text{sim}(\mathbf{z}_i, \mathbf{z}_k)}{\tau}\right)}$$

דברים שלא עובדים בSSL

1. ללמד לחזות את הframe הבא. זה לא עובד כי:
- אי אפשר לחזות את העתיד - $L2$ נוטה למצע את התמונות, וזה לא איך שהעולם נראה.
 - אם יש לנו רק $L2$, אז כל תנועה קטנה בתחזית נותנת loss מאוד גדול.

¹Self-Supervised Learning

למידת ייצוגים עם "weak" supervision

לפעמים יש לנו supervision אבל הוא מאוד חלש. בד"כ כשהתיוגים הם קורולציות מהאינטרנט. למשל: שתי תמונות הן דומות אם אנשים שחיפשו אותו דבר לוחצים על שתייהן.

מחפשים triplet relative similarity - פונקציה $f(x) : R^D \rightarrow D^d$. רוצים שהמרחק לדוגמה חיובית יהיה יותר קטן מהמרחק לדוגמה השלילית:

$$\text{Distance}(f(\text{query}), f(\text{pos})) < \text{Distance}(f(\text{query}), f(\text{neg}))$$

הגרדיאנט שאנחנו מחפשים דוחף את הדוגמה השלילית ומושך את הדוגמה החיובית. אפשר להכליל את הדבר הזה למשהו מוליט-מודאלי. נניח שיש לנו אינפורמציה על דברים שמופיעים ביחד. למשל - אם בסרט רואים כלב ושומעים את הנביחה שלו, נרצה שהתמונה והסאונד יהיו קרובים במיפוי שלהם למרחב הסמנטי. חשוב לבחור כאן negative קשים. אבל אסור להתעקש על דוגמאות שאנחנו לא מצליחים לסווג ולהשתמש בהן שוב ושוב, כי אז נלמד את הרעש בדוגמאות הספציפיות האלה.

Diffusion Models

זו דרך לייצר תמונות.

- הרעיון: 1. לוקחים תמונה של חתול.
- מוסיפים בכל שלב עוד ועוד רעש, עד שנשארים רק עם רעש.
- מאמנים מודל שילמד זוגות עוקבים, וילמד לנקות את הרעש ולהגיע לתמונה הקודמת.
- מפעילים את המודל על רעש אקראי - זה מוביל לתמונה חדשה.

החיסרון: זה מאוד כבד, ודורש הרבה צעדים. יש טריקים לגרום לזה להיות יותר יעיל. בעצם יש תהליך שמוסיף רעש אקראי בכל צעד. מתחילים ב- x^0 (שאינו רעש) ומפעילים פונקציה סטוכסטית q שמגרילה איזה רעש נוסף ל- x^t . q היא פונקציה גאוסיאנית:

$$q(x^{t+1}|x^t) = \mathcal{N}(\sqrt{1-\beta_t}x^t, \beta_t I)$$

כאשר β_t הם היפר-פרמטרים.
לחלופין:

$$x^{t+1} = \sqrt{1-\beta_t}x^t + \beta_t \epsilon \quad \epsilon \sim \mathcal{N}(0, I)$$

נשים לב שכשסוכמים התפלגות נורמלית מקבלים התפלגות נורמלית (עם פרמטרים אחרים), ולכן:

$$q(x^T|x^0) = \mathcal{N}(\sqrt{\alpha_T}x^0, \sqrt{1-\alpha_T}I) \quad \alpha_t = \prod_{s=0}^t (1-\beta_s)$$

ולכן אפשר גם לנקות במרווחים. דרך אחרת להסתכל על זה - המודל לומד למפות מהתפלגות גאוסיאנית להתפלגות המקורית. כדי למנוע model collapse (שהמודלים תמיד יתנו אותן תמונות) בכל שלב של ניקוי המודל מוסיף קצת רעש חדש משל עצמו.

ארכיטקטורה

אנחנו לא עובדים במרחב הפיקסלים. יש לנו encoder (שגם הוא רשת) שמוריד אותנו למימד הרבה יותר נמוך - למשל $32 \times 32 \times 4$, והוא נפרד מהמודל של diffusion כי הוא אומן מראש (יחד עם decoder שלו) והרבה מודלים של diffusion משתמשים באותו encoder. כדי לחבר את הטקסט לתמונה יש רכיב שנקרא cross-attention. אפשר לחשוב עליו בתור חלק ברשת שאומר לך כמה כל פיקסל בתמונה מושפע ממילים שונות בטקסט. מנרמלים אותו שיגיד לנו איזה פיקסלים מושפעים מאיזו מילה.

text-to-image של post-training

- הרבה פעמים אם מבקשים מהמודל משהו שלא מסתדר עם prior (למשל "a golden strawberry with a red crown", הוא מצייר לך את מה שהוא מכיר (תות אדום עם כתר זהוב). הפתרון הוא שאחרי כל צעד ניקוי עושים צעד תיקון שמוזיז אותנו קצת ב latent spaces כדי להקטין את loss של overlap.
- צעדי הניקוי - מגיעים מה prior.
- צעדי התיקון - מגיעים מהבקשה.
- דוגמה אחרת - ספירה של אובייקטים. קשה ללמוד מודל לספור. הפתרון הוא להסתכל על cross-attention maps, ולראות כמה blobs יש. ואז אפשר להוסיף צעד תיקון שמתקן את מספר ה blobs.
- עוד משימה - הוספה של פרטים לתמונה קיימת. כאן צעדי התיקון צריכים להכריח את המודל לשמור על התמונה החדשה קרובה לתמונה המקורית.
- הפרדת ווידאו ל layer. זו בעיה קשה, כי איך קובעים מה שייך למה? למשל צל - שייך לרקע או לאובייקט? באמצעות self-attention אפשר להבין איזה פיקסלים משפיעים אחד על השני - המודל בכל מקרה חייב ללמוד על הקשר בין הצל (או ההשתקפות) לאובייקט.